GemStone Database Explorer

A Python/Flask web application for browsing and inspecting objects in a GemStone/S object database. It started as a port of maglev-database-explorer-gem, but the current app now includes a fuller browser/debugger toolset on top of gemstone-py.

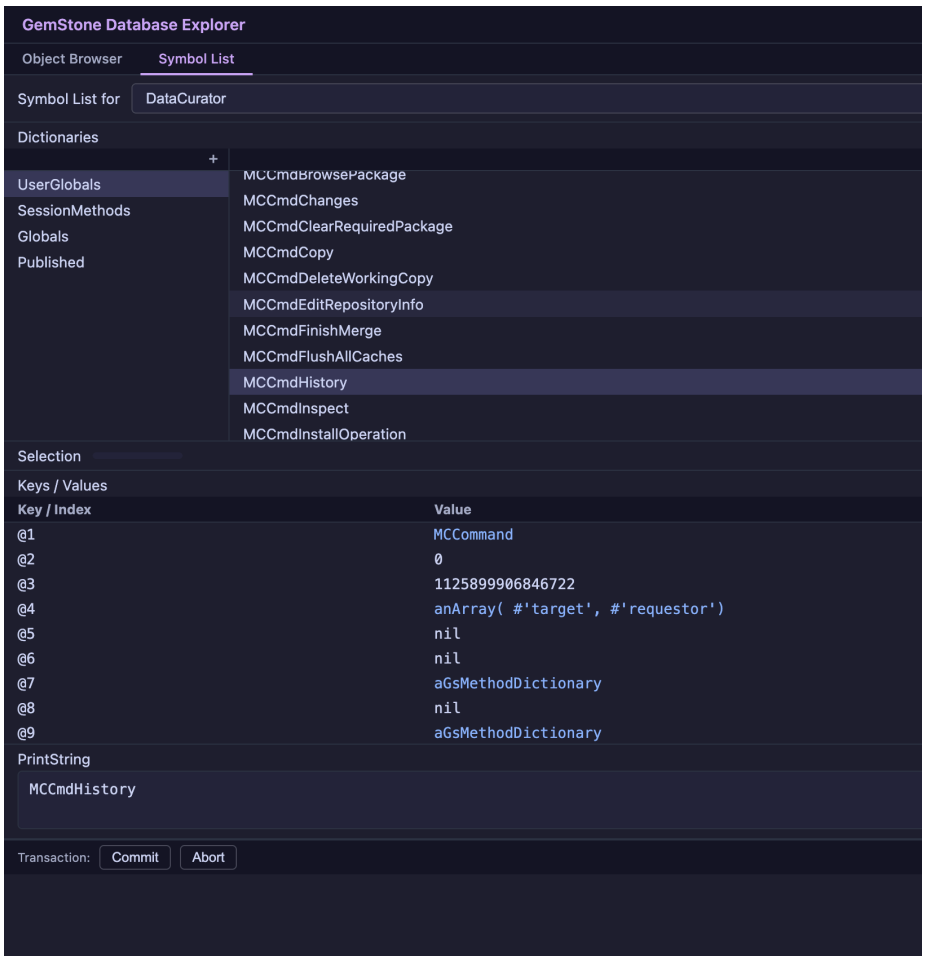

Figure 1: GemStone Database Explorer

Features

- Object inspectors with draggable object chips, linking arrows, eval, transaction controls, and backend-driven tabs
- Workspace windows that can evaluate code, open linked inspectors, and auto-open the debugger on halts

- Full Class Browser with dictionary/class/protocol/method panes, compile, file-out, structure editing, helper windows, and inspect actions
- Codegen Explorer for live class/method discovery, selector selection, JSON metadata export, and gemstone-py wrapper preview
- Helper windows for method queries, hierarchy, and versions, including load/open/inspect flows
- Debugger windows with halted-thread summaries, stack frames, locals, thread-local storage, step/proceed/trim, and execution-point highlighting
- Symbol List Browser for users, dictionaries, entries, and value inspection
- MagLev-style custom object tabs such as `Attributes` for record-like objects
- Layout persistence, taskbar/task grouping, related-window actions, splitters, filters, and keyboard navigation
- About, Status Log, and Connection windows plus `/healthz`, `/diagnostics`, `/connection/preflight`, and support-bundle export for build/runtime visibility and diagnostics capture
- Mock and live Playwright UI suites in addition to Python route/object/session tests

Requirements

- GemStone/S 64 3.x with accessible GCI libraries
- Python 3.11+
- gemstone-py

Installation

The package is available on PyPI at python-gemstone-database-explorer 1.0.1:

```
python3 -m pip install python-gemstone-database-explorer
```

For local development:

```
git clone https://github.com/unicompute/python-gemstone-database-explorer
cd python-gemstone-database-explorer
python3 -m venv .venv
.venv/bin/pip install -e .
```

Configuration

Set the GemStone connection environment before starting the app:

Variable	Description	Example
`GEMSTONE`	Path to the GemStone installation	`/opt/gemstone/GemStone64Bit3.7.5-arm64.Darwin`
`GS_USERNAME`	GemStone login username	`DataCurator`

Variable	Description	Example
`GS_PASSWORD`	GemStone login password	`swordfish`
`GS_STONE`	Stone name	`gs64stone`
`GS_HOST`	Host running the Stone/NetLDI	`localhost`
`GS_NETLDI`	NetLDI service name or port	`50377`

Depending on your platform/install, you may also need the native library path exported, for example:

```
export GS_LIB=/opt/gemstone/product/lib
```

If your Stone is named `seaside`, set `GS_STONE=seaside` before starting the app. The app also accepts `GS_STONE_NAME=seaside` as a compatibility alias, but the underlying client library reads `GS_STONE`.

See docs/configuration.md for the complete environment details.

Usage

Set the connection environment, choosing the stone name that matches your local GemStone installation:

```
# Choose one:
export GS_STONE=seaside
# export GS_STONE=gs64stone

export GS_HOST=localhost
export GS_NETLDI=50377
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish

.venv/bin/python-gemstone-database-explorer
```

For a fully explicit local run:

```
GEMSTONE=/Users/tariq/GemStone64Bit3.7.5-arm64.Darwin \
GS_USERNAME=DataCurator \
GS_PASSWORD=swordfish \
GS_STONE=gs64stone \
GS_HOST=localhost \
GS_NETLDI=50377 \
DYLD_LIBRARY_PATH=/Users/tariq/GemStone64Bit3.7.5-arm64.Darwin/lib \
/Users/tariq/src/python-gemstone-database-explorer/.venv/bin/python -m gemstone_p.cli --host
```

If installed from PyPI instead of the local editable checkout:

```
python-gemstone-database-explorer --host 127.0.0.1 --port 9292
```

For local editable installs, the command is:

```
.venv/bin/python-gemstone-database-explorer
```

Options:

```
--host HOST    Bind host (default: 127.0.0.1)
--port PORT    Port (default: 9292)
--debug        Enable Flask debug mode
```

Then open `http://127.0.0.1:9292/` in a browser.

If startup fails because the wrong Stone name or local monitor target is configured, use the taskbar `Connection` window. It shows the effective target, the local `gslist -lcv` probe when available, and a copyable shell fix such as `export GS_STONE=seaside`.

Codegen Explorer

The Codegen Explorer is a live selection surface for `gemstone-py` wrapper generation. It discovers dictionaries, classes, protocols/categories, selectors, and method source from the connected Stone, while keeping the generation rules in `gemstone-py`.

Current workflow:

- filter classes and selectors from live metadata
- filter selectors by protocol/category and inspect method source before selecting
- add instance-side fields, methods, and class-side methods to a selection
- edit generated Python method names, argument names, and return annotations before previewing
- export normalized selection JSON and import that JSON back into the UI later
- preview the generated Protocol draft and wrapper package without writing files

Testing

Python

```
.venv/bin/python -m pytest -q
```

UI

Install the Playwright dependency once:

```
npm install
```

Run the deterministic mock-backed browser suite:

```
npm run test:ui
```

That command never connects to a live GemStone runtime. It is the default regression lane for local UI work and CI.

Run only the live UI suite:

```
export GEMSTONE=/opt/gemstone/GemStone64Bit3.7.5-arm64.Darwin
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
export GS_STONE=gs64stone
export GS_HOST=localhost
export GS_NETLDI=50377
```

```
npm run test:ui:live
```

To run the mock suite first and then the live suite when the required GemStone environment is present:

```
npm run test:ui:all
```

The live suite starts the real Flask app on `127.0.0.1:4192` and covers startup browsing, Codegen Explorer metadata discovery, debugger flow, and a transactional Class Browser write flow that aborts its changes before the test ends.

Session Soak

For longer operational pressure against the shared session broker, run:

```
.venv/bin/python -m gemstone_p.session_soak --workers 8 --iterations 100 --channels 4
```

This is not a throughput benchmark. It is a maintenance tool for exercising channel reuse, login/logout churn, write-channel cleanup, and broken-session recovery against a real Stone.

Documentation

- Configuration
- Architecture
- API Reference
- Live UI Maintainer Notes
- Changelog

License

MIT